

Learning Report

1. Why ViT?

CNNs have been the go-to for vision tasks for a long time, but they have a fundamental limitation: they only look at small local patches through their convolutional filters. To get a sense of the big picture, you need to stack a lot of layers. Vision Transformers (ViTs) take a completely different approach. Instead of sliding filters across the image, they chop the image into patches and let every patch attend to every other patch from the very first layer. This means the model can pick up on long-range relationships right away -- something CNNs really struggle with.

Another practical benefit is scalability. ViTs get better and better as you throw more data and compute at them, and at large scale they consistently outperform CNNs. They're also flexible: you can use essentially the same Transformer backbone for images, text, audio, or even multi-modal inputs, which makes them a great fit for projects like ours where we might want to combine imaging data with other measurements down the line.

2. How Image Patchification Works

Transformers expect a sequence of tokens as input, not a grid of pixels. So the first thing ViT does is split the image into a grid of small, non-overlapping square patches. For example, if you have a 224x224 image and use patch size 16, you get $14 \times 14 = 196$ patches, each one a little 16x16 tile.

Under the hood this is implemented as a single convolution with kernel size and stride both set to the patch size. Each patch gets flattened and projected into a vector, and from that point on the model treats these vectors exactly like word tokens in a language model. It's a surprisingly simple trick that makes the whole Transformer machinery work for images.

3. Patch Embeddings

Once the image is chopped into patches, each patch is projected into a fixed-size vector through a learnable linear layer (the convolution mentioned above). In DINOv2-base, each patch becomes a 768-dimensional vector. These are the patch embeddings.

Because the projection weights are learned during training, the model figures out on its own what low-level features to extract -- edges, textures, colour gradients, that sort of thing. With a patch size of 16 and a 224x224 input, we end up with $14 \times 14 = 196$ patch embeddings that get fed into the Transformer.

4. Positional Embeddings

Self-attention doesn't have any built-in notion of spatial order. If you shuffled all the patch embeddings, the attention mechanism wouldn't know the difference. That's obviously a problem for images, where position matters a lot.

The fix is simple: add a learnable positional embedding to each patch embedding. Each position in the sequence gets its own 768-d vector, and these are added element-wise to the patch embeddings. During training, the model learns that position 0 corresponds to the top-left corner, position 195 to the top-right, and so on. Interestingly, if you visualise these learned positional embeddings, they form a smooth 2-D grid pattern -- the model essentially recovers the spatial layout on its own.

5. CLS Token

Before the patch sequence enters the Transformer, a special [CLS] token is prepended to the front. This token

doesn't correspond to any particular part of the image. Instead, it acts as a learnable aggregator: as the sequence passes through each Transformer layer, the CLS token attends to all the patches and gradually accumulates a summary of the whole image.

After the final layer, the CLS token's output is a single 768-d vector that captures the overall content of the image. This is what you'd use for tasks like classification or retrieval, where you need one compact representation per image.

6. ViT Transformer Architecture

The core of ViT is a stack of identical Transformer blocks. DINOv2-base uses 12 of them. Each block has two main parts: a multi-head self-attention layer and a feed-forward network (FFN), each wrapped with a residual connection and layer normalisation.

In the attention layer, every token computes attention scores against every other token, then uses those scores to create a weighted mix of all the other tokens' representations. There are 12 attention heads running in parallel, each one focusing on different aspects of the data. The FFN that follows is just two linear layers with a GELU activation in between, expanding the dimension to 4x (3072) and projecting back down to 768.

The full model has about 86 million parameters. Not tiny, but quite manageable by modern standards.

7. CLS vs Patch Tokens

The Transformer output gives you two different kinds of representations, and it's important to understand when to use which.

The CLS token (at index 0) is a global summary of the image. If you want to classify a galaxy as spiral vs. elliptical, or if you want to do nearest-neighbour retrieval, this is what you'd use. It's one vector, it's compact, and it captures the big picture.

The patch tokens (indices 1 through 256) are spatially localised. Each one tells you about a specific region of the image. These are what you'd use for segmentation, or for visualising which parts of the galaxy the model is paying attention to. In our notebook, we use PCA on these patch tokens to create a colour-coded spatial map of DINOv2's features.

8. What DINOv2 Is

DINOv2 is a self-supervised model from Meta AI (2023) that learns visual representations without any human labels at all. It uses two training objectives working together. The first is self-distillation: a student network tries to match a teacher network's outputs on different augmented views of the same image. The teacher is a slowly-updated copy of the student, which keeps things stable.

The second objective is masked image modelling: some patches are randomly hidden, and the student has to predict what the teacher would have produced for those patches. This forces the model to learn both global semantic understanding and local spatial reasoning.

The result is a set of features that are shockingly general. Without any fine-tuning, DINOv2 matches or beats specialised supervised models on benchmarks ranging from image classification to depth estimation. It was trained on 142 million curated images.

9. What DINOv2 Outputs

When you pass a 224x224 image through DINOv2-base, you get a tensor of shape (batch, 257, 768). That's 1 CLS token + 256 patch tokens, each 768 dimensions. This is the `last_hidden_state`. You can also request intermediate hidden states from all 12 layers if you need multi-scale features.

Importantly, there's no classification head -- the model just gives you raw embeddings. You can add whatever head you want on top: a linear layer for classification, a k-NN for retrieval, an MLP for regression. The embeddings themselves are already highly informative because of the self-supervised pretraining.

10. How DINOv2 Will Be Used in This Project

In our project, we use DINOv2 as a frozen feature extractor -- no fine-tuning, just extracting the pretrained representations. The dataset is Galaxy Zoo 10 from the Multimodal Universe collection: 160x160 RGB cutouts of galaxies from DESI imaging.

The pipeline works like this: we take each galaxy image, stretch the pixel values to a nice [0, 255] range using percentile normalisation, resize to 224x224, and run it through DINOv2. We then look at two things.

First, we take the patch tokens, reduce them from 768-d to 3-d using PCA, and map them to RGB colours on a 16x16 grid. This gives us a spatial feature map that shows what structural features (bulge, disk, spiral arms, halo) the model is picking up on for each galaxy.

Second, we extract the CLS token for 100 galaxies and project them to 2-D using PCA and t-SNE. Even without any training on galaxy data, we can see that DINOv2 naturally groups similar morphologies together in the embedding space -- spirals end up near spirals, ellipticals near ellipticals. The next step will be to train a lightweight classifier on top of these embeddings using the gz10_label annotations.